

COM1B/01-04373/2021

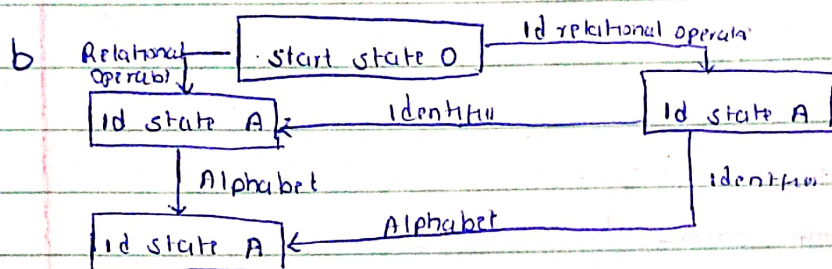
INDIAZI ALVIN WESLEY

BCS 324

COMPILER CONSTRUCTION AND DESIGN

CAT 2 (ANSWERS)

- a) Next-Use information helps optimize memory access patterns and minimizes the number of instructions executed. The compiler can make informed decisions about register allocation, scheduling and memory management.



c Quadruples

$T_1, T_2, T_3, T_4 \rightarrow$ temporary variables.

Op $\rightarrow$ Operations

a, b, c $\rightarrow$ variables.

(ADD, a, b, T_1) // $T_1 = a + b$

(ADD, T_1 , c, T_2) // $T_2 = T_1 + c$

(NEG, T_2 , T_3) // $T_3 = -T_2$

(MUL, T_1 , T_3 , T_4) // $T_4 = T_1 * T_3$

o indirect T_1 plus:

(ADD, a, b, T_1) // $T_1 = a + b$

(ADD, T_1 , c, T_2) // $T_2 = T_1 + c$

(NEG, T_2 , T_3) // $T_3 = -T_2$

(MUL, T_1 , T_3) // $T_1 * T_3$

d Translation Scheme

i) $E \rightarrow E_1 \text{ relop } E_2$

{ backpatch (E_1 .trueList, E_2 .quad)

E .trueList = E_2 .trueList

E .falseList = merge (E_1 .falseList, E_2 .falseList) }

ii) $E \rightarrow \text{not } E_1$

{ E .trueList = E_1 .falseList

E .falseList = E_1 .trueList }

iii) $E \rightarrow (E_1)$

```
{ E.trueList = E1.trueList  
  E.falseList = E1.falseList  
}
```

iv) $E \rightarrow id$

```
{ E.trueList = makeList (nextQuad)  
  E.falseList = makeList (nextQuad + 1)  
  emit ("if id rlop goto -")  
  emit ("goto -")  
}
```

Attributes used:

$E \rightarrow$ Boolean expression being evaluated

$E_1, E_2 \rightarrow$ sub-expressions within E

rlop $\rightarrow$ relational operators

trueList : list of true branches in the expression

falseList : list of false branches in the expression

id : Identifiers within the expression.

e) $a = b + 1;$

$\rightarrow t_1 = b + 1;$

$a = t_1;$

$x = y + 3$

$\rightarrow t_2 = y + 3$

$x = t_2$

$y = a / b$

$\rightarrow t_3 = a / b$

$y = t_3$

$a = b + c$

$\rightarrow t_4 = b + c$

$a = t_4$

F. Graph Coloring $\rightarrow$ models register allocation as a graph coloring problem

- Linear Scan $\rightarrow$ Allocates registers as variables are encountered during single pass over the code.
- Optimistic Coloring $\rightarrow$ Assign registers optimistically based on heuristic such as first fit.
- Hybrid Approach $\rightarrow$ Leveraging the strengths of each strategy to achieve better overall performance
- Handling Spills $\rightarrow$ Allocates registers by balancing it with spill handling
- Optimizations $\rightarrow$ loop unrolling, function inlining and common subexpression can reduce number of live variables and alleviate register pressure

G. Abstraction - Capturing the essential semantics of the source code while being independent of the target machine architecture

Portability - Code should be portable across different target architectures

Optimization - Improves efficiency and performance of generated code

* Challenges include

Data Locality - Optimisation is crucial for improving on architectures

Synchronization overhead - Mechanisms such as locks and barriers to co-ordinate concurrent execution are required by parallel architectures

Complexity $\rightarrow$ need for additional complexity for code optimization in modern parallel architectures

i) Parser Generators $\rightarrow$ Automate construction of parsers from formal grammars

• Syntax Directed Translation Engines are responsible for translating input source code into intermediate representations or target code while simultaneously performing semantic actions based on the syntactic structure of the input

• Scanner Generators $\rightarrow$ Automate the construction of lexical analyzers for programming languages.